

adk-models-go

Consolidation specification

Implementation specification • 8 September 2026 • Revision 0.4

Status: Behaviour decisions, public-interface structure and the delivery requirements in this document are approved by Joel Beach. Unspecified field-level semantics still require a decision before affected implementation. Publishing is outside scope.

1. Purpose and scope

Combine adk-anthropic-go, adk-openai-go and adk-vercel-go into one repository named **adk-models-go**, one Go module and one shared release version. Reduce duplicate code while making shared behaviour explicit.

Preserve three adapters: **Anthropic Messages**, **OpenAI Responses** and **native Vercel**. The caller selects the adapter explicitly. The library must not select or switch request formats automatically.

2. Binding change-control rule

All behaviour not explicitly changed in this specification must remain unchanged. This includes request conversion, tool calls, streaming, file handling, usage reporting and retries.

If implementation exposes an ambiguity, conflicting requirement or additional behaviour change, stop the affected work and obtain a decision. Do not guess, silently resolve the conflict or treat an implementation convenience as approval.

3. Code consolidation

- Share equivalent model-family detection, reasoning mappings, Vercel configuration and validation, metadata fields and other behaviourally equivalent helpers.
- Keep format-specific conversion and processing separate wherever needed to preserve behaviour.
- Removing duplicate code must not silently change request payloads, response interpretation or errors.

Terminology

Model family means the model developer or family: OpenAI, Anthropic, Gemini or Z.ai. A **serving provider** runs the model. A **route** is the connection used to reach it. Shared reasoning mappings belong to the model family, not to the serving provider or route.

Model identity

4. Canonical and request names

Name	Role
Canonical model name	Stable model identity; determines the model family and its mappings.
Request model name	Exact identifier sent to the endpoint. Defaults to the canonical name when omitted.

Family detection ignores case and surrounding spaces. This detection normalisation does not rewrite an explicitly supplied request name.

Recognised canonical patterns

Family	Patterns
OpenAI	gpt-*, o1, o1-*, o3, o3-*, o4, o4-*
Anthropic	claude-*
Gemini	gemini-*
Z.ai	glm-*

Canonical names are unqualified. Endpoint prefixes such as openai/ belong in the request name. Reject unrecognised canonical names. Do not add a family override or an exhaustive model-ID catalogue.

Request-name mismatch checks

- Recognise request families from the approved bare patterns or the prefixes openai/, anthropic/, google/ and zai/.
- Reject a recognised request family that differs from the canonical family.
- Reject a request name whose recognised prefix and recognised model-name portion identify different families.
- Skip the cross-check for an unrecognised request alias. Always use the canonical family for mappings.

Canonical	Request	Result
gpt-example	openai/gpt-example	Accept
gpt-example	anthropic/claude-example	Reject
gpt-example	my-deployment	Accept; alias unverified
gpt-example	openai/claude-example	Reject

Recognising a model family does not establish that every adapter or endpoint supports it.

Reasoning contract

5. Shared levels and mappings

Use `genai.ThinkingLevel`. Expose shared constants **ThinkingLevelXHigh = "XHIGH"** and **ThinkingLevelMax = "MAX"**, both typed as `genai.ThinkingLevel`, for backend code and model routes. HIGH remains distinct.

Requested	OpenAI	Anthropic effort	Gemini	Z.ai
MINIMAL	none	low; disabled	MINIMAL	low
LOW	low	low; adaptive	LOW	low
MEDIUM	medium	medium; adaptive	MEDIUM	high
HIGH	high	high; adaptive	HIGH	max
XHIGH	xhigh	xhigh; adaptive	HIGH	max
MAX	xhigh	max; adaptive	HIGH	max

In the Anthropic column, disabled and adaptive refer to thinking mode. For explicit Z.ai levels, thinking remains enabled. Each adapter encodes these shared family decisions in its own request format.

Defaults and validation

- Without a supplied or explicitly caller-configured level, omit reasoning-level settings and use the model default. Do not introduce an implicit library effort default.
- For Anthropic, an unset level omits both thinking and effort settings. Gemini standard levels pass through unchanged; custom XHIGH and MAX map to HIGH.
- Use family mappings only. Do not maintain model-specific capability exceptions. Return provider or route errors for rejected settings; do not retry at a different effort level.
- Reject explicit ThinkingBudget values. Do not translate them into effort levels or silently discard them.

Summary visibility and state

`IncludeThoughts` controls whether reasoning summaries are returned. It must not enable, disable or change reasoning effort. Preserve opaque reasoning state required for later turns even when summaries are hidden.

6. OpenAI response storage

Always send **store: false** for OpenAI. ADK owns conversation history; preserve the reasoning state needed for ADK-managed continuation. Do not introduce a `store: true` option.

This is separate from Vercel retention. Allowing retention does not enable OpenAI response storage.

Configuration and runtime behaviour

7. VercelConfig

All Vercel-specific settings belong on VercelConfig. This includes routing, retention, gateway-managed caching, gateway-specific provider options and other existing Vercel-only controls.

Do not expose gateway-managed caching as a general caching mode that can be selected independently of Vercel configuration.

Retention and option conflicts

- Allow retention by default. Callers explicitly request zero data retention through VercelConfig.
- Send the configured requirement to Vercel and let the gateway determine support. Propagate its errors. Do not add local capability checks or weaken the requirement after an error.
- Reject raw provider options that conflict with typed settings. Neither may silently replace conflicting values from the other.

8. Prompt caching and unsupported features

- Leave caching controls unset unless explicitly requested. Provider-managed caching may still occur.
- Keep model-family caching settings separate from gateway-managed caching.
- Ignore unsupported optional features, including cache breakpoints. Preserve existing supported placement, modes and cache lifetimes unless another approved rule changes them.

The ignore rule does not override budget rejection, raw/typed conflict errors, reasoning mappings or explicit retention requirements. It does not authorise stripping settings and retrying after a provider error.

9. Metadata

Expose standard metadata fields plus separate raw provider metadata. Preserve existing supported usage, cost, routing, response identity and cache-write information. Do not discard information currently exposed by an adapter.

The caller controls logging and tracing. The library must not automatically log returned metadata or attach it to traces.

10. Clients, retries and errors

Anthropic and OpenAI retain caller-built SDK clients, preserving control over authentication, endpoints, HTTP behaviour and SDK options. Native Vercel accepts an optional HTTP client and defaults to `http.DefaultClient`.

Preserve SDK-configured and adapter-specific retries, including conditions and limits. Share retry code only where behaviour is equivalent. Preserve provider-error meaning; do not add automatic format switching or fallback changes to reasoning, retention or caching.

Verification and approval

11. Acceptance evidence

Retain existing adapter tests. Change expected behaviour only where this specification explicitly requires it. Add focused coverage for:

- Family patterns, case and whitespace handling, request-name defaults and mismatch checks.
- Every reasoning mapping across applicable adapters, unset reasoning, and token-budget rejection.
- Summary visibility and opaque-state preservation; OpenAI store: false.
- Retention allowed by default, explicit zero-retention requests and propagation of gateway errors.
- Conflicting raw and typed settings; unsupported cache controls being ignored.
- Standard and raw metadata preservation, client ownership and existing retry behaviour.

Distinguish local payload correctness from remote support. Successful conversion is not proof that a provider accepts a request. Do not claim authenticated integration coverage unless it was actually performed.

12. Approved draft PR scope

- Create the combined library and port existing tests. Implement the approved behaviour changes and add shared interface and mapping tests.
- Include the final specification in the new repository.
- The combined-library PR remains library-only. A separate backend migration must remove the Luna workaround; README deprecation notices in all three old adapters are also required delivery work (section 18). Publishing releases and archiving repositories remain outside scope.

The interface structure below is approved. Do not silently invent unresolved field types, defaults or conflict semantics. Raise any remaining field-level ambiguity before implementing the affected part.

Reference basis

Behavioural baseline: the backend-pinned releases `adk-anthropic-go/v3 v3.0.0`, `adk-openai-go v0.1.0` and `adk-vercel-go v0.1.0`. This document records approved design decisions; it is not a claim of universal model or route compatibility.

Supporting references checked during specification discussions:

[Anthropic: effort and thinking controls](#)

[Z.ai: GLM-5.3 effort levels](#)

[OpenAI: conversation state](#)

Approved public interface

13. Module, version and packages

Module: `github.com/Alcova-AI/adk-models-go`

Initial release: v0.1.0. This is the intended version; publication is outside the draft PR scope.

Package	Responsibility
<code>adkmodels</code> (module root)	Shared configuration, reasoning constants, metadata types and accessors.
<code>anthropic</code>	Anthropic Messages adapter.
<code>openai</code>	OpenAI Responses adapter.
<code>vercel</code>	Native Vercel adapter.
<code>internal/...</code>	Shared implementation with no public compatibility promise.

All packages ship under one version. Each adapter constructor returns `(model.LLM, error)`, using ADK's existing model interface.

14. Adapter constructors

Anthropic: `anthropic.NewModel(anthropic.Config{Client: sdkClient, Model: modelConfig})`

OpenAI: `openai.NewModel(openai.Config{Client: sdkClient, Model: modelConfig})`

Native Vercel: `vercel.NewModel(vercel.Config{APIKey: apiKey, BaseURL: baseURL, HTTPClient: httpClient, Headers: headers, Model: modelConfig})`

Anthropic and OpenAI clients retain caller-controlled authentication and endpoint selection. Native Vercel retains its connection settings and optional HTTP client.

15. Shared ModelConfig

Field	Approved type
<code>CanonicalModel</code>	<code>string</code>
<code>RequestModel</code>	<code>string</code>
<code>DefaultMaxOutputTokens</code>	<code>int32</code>
<code>Reasoning</code>	<code>ReasoningConfig</code>
<code>PromptCaching</code>	<code>PromptCachingConfig</code>
<code>Vercel</code>	<code>*VercelConfig</code>

Configuration and metadata interface

16. Configuration responsibilities

- ReasoningConfig holds an optional caller-selected default level and existing family-specific reasoning controls.
- PromptCachingConfig holds model-family caching controls.
- VercelConfig holds all gateway-specific behaviour, including routing, retention, gateway-managed caching and gateway provider options.
- Direct adapters use Vercel: nil when no gateway extension is required. Native Vercel uses default gateway settings when the field is nil.

Family-specific controls

Keep supported controls that are not interchangeable in explicit nested configuration:

- Anthropic caching: cache lifetimes and distinct boundaries.
- OpenAI caching: modes, cache key and boundary rules.
- OpenAI reasoning: existing context, mode and summary controls.

These controls must not override approved shared effort mappings. No generic raw-options mechanism may bypass conflict validation.

17. Shared response metadata

Expose one accessor:

metadata, ok := adkmodels.MetadataFromResponse(response)

Use one shared metadata type containing:

- Response ID and cache-write input tokens.
- Gateway generation ID, resolved provider and model identity.
- Gateway model and provider attempt counts.
- Optional cost in USD and separate raw provider metadata.

ADK standard token-usage fields remain the standard usage interface. Preserve additional metadata exposed by existing adapters.

Approval boundary

The package layout, constructor shapes, ModelConfig fields, nested responsibilities and metadata contents are approved. Exact nested field definitions and any unspecified semantics must follow preserved behaviour where unambiguous. Any ambiguity requires a decision; it is not delegated implementation discretion.

Migration, notices and licence

18. Required follow-up delivery

The combined-library draft PR remains library-only. The following related migration and documentation changes are now required delivery work, tracked separately rather than omitted from the consolidation.

Luna production regression: PR #2322

[Alcova-AI/alcova-backend PR #2322](#) is the fixed reference for the temporary workaround. Joel confirmed it is in the merge queue and its diff will not change; library work need not wait for it to merge.

The production failure was an explicit MINIMAL request from email query expansion being sent as minimal, which Luna rejected. Query expansion then fell back to the original query. The temporary wrapper changes MINIMAL to LOW; the approved library mapping instead sends none.

- In the same backend migration that adopts `adk-models-go`, remove `lunaReasoningModel` and its `model_factory` hook. Do not retain or replace it with another Luna-specific mapping wrapper.
- The shared OpenAI family mapping must own MINIMAL to none. Keeping the workaround would change the request to LOW before the library sees it.
- Retain `TestProductionV2LunaWireRouting` and `TestLunaReasoningPreservesCallerConfiguration` coverage, adapting their assertions to the new library instead of testing the removed wrapper.
- Assert that explicit MINIMAL produces none in both `reasoning.effort` and `providerOptions.openai.reasoningEffort` on the production Vercel Responses route. Check direct Responses behaviour through the same family mapping.
- Verify that route defaults, other explicit levels, caller request/configuration objects, `IncludeThoughts`, token limits and model/route attribution are preserved except for approved mapping changes.

Before claiming the production failure is resolved, verify that the production-equivalent Luna route accepts the resulting request. Local payload tests alone establish conversion correctness, not remote acceptance. Record any missing remote verification explicitly. Do not deploy as part of draft PR creation.

Licence and attribution

Keep **Apache License 2.0**, matching the existing projects. Include the existing licence text in the new repository, preserve source copyright notices, and carry forward applicable third-party notices and attribution. Do not relicence imported code.

Deprecation notices in the old adapters

Add prominent README deprecation notices to `adk-anthropic-go`, `adk-openai-go` and `adk-vercel-go`. Each notice must point to github.com/Alcova-AI/adk-models-go and its corresponding anthropic, openai or vercel package.

State that the adapter is superseded by the combined library and direct readers to the new README for migration guidance. Distinguish a draft PR from an available release; do not claim a published replacement before one exists. Preserve existing releases and repository access. README notices do not authorise archival or deletion.

Approved implementation clarifications

19. Direct model-family boundary

Direct Anthropic Messages supports Anthropic-family models. Direct OpenAI Responses supports OpenAI-family models. Reject direct cross-family combinations locally. Cross-family requests use an explicitly configured Vercel adapter or Vercel extension; the library never switches formats automatically.

Direct Gemini remains on Google ADK's existing Gemini adapter, outside this library. Gemini through Vercel is supported by the combined library and uses the shared Gemini level mapping. Family recognition alone does not provide remote support for a specific route.

20. Forced Anthropic tool use

Preserve the existing Anthropic exception for forced tool use: clear thinking, and clear effort when thinking was adaptive. Apply the exception to both native Messages fields and Anthropic gateway provider options, so one representation cannot restore controls removed from the other.

This exception applies when tool use is forced, including a specific named tool. It does not change the caller's request or configuration. Other requests keep the approved family mapping.